

Variationally stable reduced basis neural operator

Yuan Qiu¹ Wolfgang Dahmen² Peng Chen¹

¹Georgia Institute of Technology

²University of South Carolina

March, 2026

Physics-informed operator learning

- Neural partial differential equation (PDE) solver: find $\mathcal{N}_\theta$ such that

$$\mathcal{N}_\theta(\mathbf{p}) = \mathbf{u}, \quad \text{subject to} \quad \mathcal{R}(\mathbf{p}, \mathbf{u}) = 0, \quad \text{for all} \quad \mathbf{p} \sim \mu,$$

where

- $\mathbf{p}$ — parameter function/array; μ — probability distribution
- $\mathbf{u}$ — state/solution function
- $\mathcal{R}(\cdot, \cdot)$ — PDE residual
- $\mathcal{N}_\theta(\cdot)$ — neural network parameterized by trainable weights θ
- Benefits
 - deployed in downstream tasks that need frequent evaluation of $\mathbf{u} = \mathbf{u}(\mathbf{p})$
 - quickly provides a good initial guess of $\mathbf{u}$ for the iterative solver
 - ...
- One way to promote physical consistency is to minimize PDE residual loss

Motivation

Stationary diffusion equation with mixed Dirichlet-Neumann boundary conditions

$$\begin{cases} -\operatorname{div}(\mathbf{p} \nabla \mathbf{u}) = f, & \text{in } \Omega, \\ \mathbf{u} = u_0, & \text{on } \Gamma_D, \\ \mathbf{p} \nabla \mathbf{u} \cdot \mathbf{n} = g, & \text{on } \Gamma_N, \end{cases}$$

The residual loss in Physics-Informed Neural Network [Raissi et al., 2019]

$$\mathcal{L}(\theta) = \|\operatorname{div}(\mathbf{p} \nabla \mathbf{u}_\theta) + f\|_{L^2(\Omega)}^2 + \lambda_1 \|\mathbf{u}_\theta - u_0\|_{L^2(\Gamma_D)}^2 + \lambda_2 \|\mathbf{p} \nabla \mathbf{u}_\theta \cdot \mathbf{n} - g\|_{L^2(\Gamma_N)}^2$$

fails to provide uniform upper and lower bounds on the true error in a proper PDE norm (is NOT variationally correct) because

$$f \in H^{-1}(\Omega), \quad u_0 \in H^{1/2}(\Gamma_D), \quad g \in H^{-1/2}(\Gamma_N).$$

Variationally correct residual loss

Find $u(\mathbf{p})$ such that

$$\mathcal{F}(u; \mathbf{p})(v) = 0, \quad \forall v \in \mathbb{V},$$

with a suitable test space $\mathbb{V}$ such that

$$\|\mathcal{F}(w; \mathbf{p})\|_{\mathbb{V}'} \simeq \|w - u\|_{\mathbb{U}}$$

with constants that are uniform in $\mathbf{p}$.

Prior work

- minimax problem with Weak Adversarial Networks (WANs) [Zang et al., 2020]
- Discontinuous Petrov Galerkin (DPG) Loss [Castillo et al., 2025]
- First Order System Least Squares (FOSLS) Loss [Bachmayr et al., 2025]
- ...

FOSLS loss [Bachmayr et al., 2025]

Step 1. (weak form) Given $f \in \mathbb{U}'$ find $u \in H^1(\Omega)$ satisfying

$$u|_{\Gamma_D} = u_0, \quad \text{and} \quad b_p(u, v) := \int_{\Omega} p \nabla u \cdot \nabla v \, dx = f(v) + \langle g, v \rangle_{\Gamma_N}, \quad v \in \mathbb{U},$$

where $\mathbb{U} := \{u \in H^1(\Omega) : u|_{\Gamma_D} = 0\}$.

Step 2. (lift boundary data) Solve two auxiliary parameter-independent problems

For $u|_{\Gamma_D} = u_0$: find $w \in H^1(\Omega)$ such that

$$(\nabla w, \nabla v)_{\Omega} = 0, \quad v \in H^1(\Omega), \quad w|_{\Gamma_D} = u_0.$$

For $\langle g, v \rangle_{\Gamma_N}$: find $q \in \mathbb{U}$ such that

$$(\nabla q, \nabla v)_{\Omega} = \langle g, v \rangle_{\Gamma_N}, \quad v \in \mathbb{U}.$$

We set $z := \nabla q$, and write $u = u^\circ + w$.

FOSLS loss [Bachmayr et al., 2025]

Step 3. We assume f to be flux-free along the Neumann boundary;

$$f(v) = (f_2, v)_\Omega - (f_1, \nabla v)_\Omega, \quad v \in \mathbb{U}.$$

where $f_2 \in L_2(\Omega)$ and $f_1 \in L_2(\Omega; \mathbb{R}^d)$.

The equivalent weak formulation becomes

$$(\mathfrak{p} \nabla(u^\circ + w) - z + f_1, \nabla v)_\Omega = (f_2, v)_\Omega, \quad v \in \mathbb{U}.$$

Introducing the auxiliary flux variable

$$\sigma^\circ := \mathfrak{p} \nabla(u^\circ + w) - z + f_1 \in \Sigma := \{\tau \in H(\operatorname{div}) : \tau \cdot n = 0 \text{ on } \Gamma_N\},$$

we obtain

$$(\sigma^\circ - \mathfrak{p} \nabla u^\circ, \tau)_\Omega = (F, \tau)_\Omega, \quad -(\operatorname{div} \sigma^\circ, v)_\Omega = (f_2, v)_\Omega,$$

where $F := \mathfrak{p} \nabla w - z + f_1 \in L_2(\Omega; \mathbb{R}^d)$ and $(\tau, v) \in L_2(\Omega; \mathbb{R}^{d+1})$.

For any approximation $[\tilde{\sigma}^\circ, \tilde{u}^\circ]$, the first-order system least-squares (FOSLS) loss is

$$\mathcal{L}([\tilde{\sigma}^\circ, \tilde{u}^\circ]; \mathfrak{p}) = \|\tilde{\sigma}^\circ - (\mathfrak{p} \nabla \tilde{u}^\circ + F)\|_{L_2(\Omega; \mathbb{R}^d)}^2 + \|\operatorname{div} \tilde{\sigma}^\circ + f_2\|_{L_2(\Omega)}^2.$$

Finite element (FE) approximations

Assume the approximation $[\tilde{\sigma}^\circ, \tilde{u}^\circ]$ is in some finite dimensional FE space $\Sigma_h \times \mathbb{U}_h$:

$(RT_k \text{ with zero Neumann BC}) \times (CG_m \text{ with zero Dirichlet BC})$

where RT_k is the Raviart-Thomas element of order k , and CG_m is the continuous Galerkin element of order m . Then the FOSLS loss can be easily computed by finite element assembly for either **neural network training** or **finite element solve**.

DoFs of $\mathbf{p}$ $\xrightarrow{NN(\theta)}$ DoFs of $\tilde{\sigma}, \tilde{u} \rightarrow \text{Assembly (DoFs of } \mathbf{p}, \tilde{\sigma}, \tilde{u}; \text{ FE basis)} \rightarrow \text{optimizer}$

DoFs of $\mathbf{p}$ $\searrow$
Guess of DoFs of $\tilde{\sigma}, \tilde{u}$ $\nearrow$
Assembly (DoFs of $\mathbf{p}, \tilde{\sigma}, \tilde{u}; \text{ FE basis}) \rightarrow \text{Newton's method}$

Neural operator (architectures)

Reduced basis neural operator (RBNO)

$$\tilde{s}(\mathbf{p}; \theta)(x) = \underbrace{(\phi_1(x), \dots, \phi_{N_h}(x))}_{\text{FE basis}} \underbrace{(\pi_1, \dots, \pi_r)}_{\text{DoFs of RB}} \underbrace{s_r(\mathbf{p}; \theta)}_{\text{RB coeff.}}$$

We choose

- POD basis for π_k 's (in the discrete $H(\text{div}) \times H^1$ norm), and
- a Convolutional Neural Network (CNN) for $s_r(\cdot; \theta)$

$$s_r(\cdot; \theta) = L_2 \circ \sigma \circ L_1 \circ \text{Flatten} \circ C_L \circ \dots \circ C_1 : \mathbb{R}^{H \times W} \rightarrow \mathbb{R}^r,$$

Pros

- Greatly reduce optimization difficulty (compared to predicting full DoFs directly)
- Much less GPU memory and computational cost for training and inference

Cons

- Need some solution snapshots to compute the POD basis
- Need much more basis functions for slow decay Kolmogorov n-widths cases

Neural operator (loss evaluation)

- When the PDE is linear, we obtain the discrete FOSLS loss

$$\mathcal{L}(\tilde{\mathbf{s}}; \mathbf{p}) = \mathbf{w}^\top W_{\mathbf{p}} \mathbf{w},$$

where $\mathbf{w} = (\tilde{\mathbf{s}}; 1) \in \mathbb{R}^{N_h+1}$ is a concatenation of the DoF $\tilde{\mathbf{s}} \in \mathbb{R}^{N_h}$ and 1.

- Since we use the POD basis, the loss becomes

$$\mathcal{L}(\mathbf{s}_r; \mathbf{p}) = \mathbf{s}_r^\top W_{\mathbf{p}}^r \mathbf{s}_r + 2\mathbf{s}_r^\top \boldsymbol{\alpha}_{\mathbf{p}}^r + \beta_{\mathbf{p}},$$

To speed up training, we precompute and store $W_{\mathbf{p}}^r$, $\boldsymbol{\alpha}_{\mathbf{p}}^r$ and $\beta_{\mathbf{p}}$ offline.

- We then train the RBNO over finite parameter samples with empirical risk

$$\frac{1}{N} \sum_{i=1}^N \mathcal{L}(\mathbf{s}_r(\mathbf{p}^{(i)}; \theta); \mathbf{p}^{(i)}).$$

Theorem (FE error $\approx$ FE residual)

Theorem 1

Assume Ω is Lipschitz and $\mathfrak{p} \in L^\infty(\Omega)$ is uniformly bounded with $0 < \alpha \leq \mathfrak{p}(x) \leq \beta < \infty$ independent of $\mathfrak{p} \in \mathfrak{P}$. Let $\Sigma_h = \text{RT}_k^\circ$ and $\mathbb{U}_h = \text{CG}_m^\circ$ on a shape-regular mesh of size h with $k \geq 0$, $m \geq 1$. Let $[\sigma^\circ, u^\circ]$ be the exact solution and $[\sigma_h^\circ, u_h^\circ]$ the Galerkin solution in $\Sigma_h \times \mathbb{U}_h$. Then, for each fixed $\mathfrak{p} \in \mathfrak{P}$, one has the *error-residual equivalence*

$$\mathcal{L}([\sigma_h^\circ, u_h^\circ]; \mathfrak{p}) \approx \|\sigma^\circ - \sigma_h^\circ\|_{H(\text{div}; \Omega)}^2 + \|u^\circ - u_h^\circ\|_{H^1(\Omega)}^2,$$

with equivalence constants depending only on α, β and the domain.

Moreover, if $\sigma^\circ \in H^{s_\sigma}(\Omega; \mathbb{R}^d)$, $\text{div } \sigma^\circ \in H^{s_{\text{div}}}(\Omega)$ with $s_\sigma, s_{\text{div}} \geq 0$, and $u^\circ \in H^{s_u}(\Omega)$ with $s_u \geq 1$, then

$$\begin{aligned} \mathcal{L}([\sigma_h^\circ, u_h^\circ]; \mathfrak{p}) &\lesssim h^{2\min(k+1, s_\sigma)} \|\sigma^\circ\|_{H^{s_\sigma}}^2 + h^{2\min(k+1, s_{\text{div}})} \|\text{div } \sigma^\circ\|_{H^{s_{\text{div}}}}^2 \\ &\quad + h^{2\min(m, s_u-1)} \|u^\circ\|_{H^{s_u}}^2. \end{aligned}$$

Theorem (quasi-optimality of RB FOSLS solution)

Theorem 2

Let $s_h(\mathbf{p}) = [\sigma_h^\circ(\mathbf{p}), u_h^\circ(\mathbf{p})] \in \mathbb{H}_h := \Sigma_h \times \mathbb{U}_h$ be the *FE FOSLS solution* and

$$s_r(\mathbf{p}) = [\sigma_r^\circ(\mathbf{p}), u_r^\circ(\mathbf{p})] = \operatorname{argmin}_{[\tau_r, v_r] \in \mathbb{H}_r} \mathcal{L}([\tau_r, v_r]; \mathbf{p})$$

be the *RB FOSLS solution* in the POD subspace $\mathbb{H}_r := \operatorname{span}\{\phi_1, \dots, \phi_r\} \subset \mathbb{H}_h$.

For any $s_r \in \mathbb{H}_r$, there holds

$$\mathcal{L}(s_r; \mathbf{p}) \approx \|s_h(\mathbf{p}) - s_r\|_{\mathbb{H}}^2 + \mathcal{L}(s_h(\mathbf{p}); \mathbf{p}).$$

In particular, the reduced FOSLS solution satisfies the quasi-optimality bound

$$\|s_r(\mathbf{p}) - s_h(\mathbf{p})\|_{\mathbb{H}} \approx \min_{t_r \in \mathbb{H}_r} \|t_r - s_h(\mathbf{p})\|_{\mathbb{H}}.$$

Theorem (error decomposition)

Theorem 3

Under the uniform boundedness assumption of neural network output, for any $\delta \in (0, 1)$, with probability at least $1 - \delta$, there holds

$$\begin{aligned} \mathbb{E}_{\mathbf{p} \sim \mu} \left[\|s(\mathbf{p}) - s_r(\mathbf{p}; \hat{\theta})\|_{\mathbb{H}}^2 \right] &\leq c^{-1} \inf_{\theta \in \Theta} \mathbb{E}_{\mathbf{p} \sim \mu} [\mathcal{L}(s_r(\mathbf{p}; \theta); \mathbf{p})] \\ &\quad + M \sqrt{\frac{\log \frac{1}{\delta}}{2N}} + CL \sqrt{\frac{2P \log \frac{eN}{P}}{N}} + \varepsilon_{\text{opt}}, \end{aligned}$$

Moreover, there exist constants C_1, C_2 , depending on FE discretization, and uniform FOSLS stability, respectively, such that the approximation error can be decomposed as follows

$$\inf_{\theta \in \Theta} \mathbb{E}_{\mathbf{p} \sim \mu} [\mathcal{L}(s_r(\mathbf{p}; \theta); \mathbf{p})] \leq C_1 \left\{ h^{2\eta} + \sum_{k>r} \lambda_k \right\} + C_2 \inf_{\theta \in \Theta} \mathbb{E}_{\mathbf{p} \sim \mu} [\|s_r(\mathbf{p}) - s_r(\mathbf{p}; \theta)\|_{\mathbb{H}}^2].$$

Here the exponent η depends on the regularity of the exact FOSLS solution.

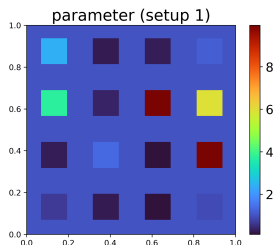
Numerical Experiments

Problem setup (stationary diffusion equation)

Strong form of the linear case:

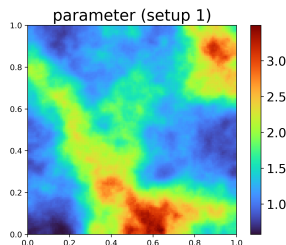
$$\begin{cases} -\operatorname{div}(\mathbf{p} \nabla u) = f, & \text{in } \Omega, \\ u = u_0, & \text{on } \Gamma_D, \\ \mathbf{p} \nabla u \cdot \mathbf{n} = g, & \text{on } \Gamma_N. \end{cases}$$

Goal: learn the parameter-to-solution map $\mathbf{p} \mapsto u$. For $\mathbf{p}$, we consider two cases:



$$\mathbf{p}(x) = \mathbf{1}_{\Omega \setminus (\cup \bar{\Omega}_i)}(x) + \sum_{i=1}^{16} 10^{\mu_i} \mathbf{1}_{\bar{\Omega}_i}(x), \quad \mu_i \sim \mathcal{U}(-1, 1)$$

(electronics heat conduction)



$$\mathbf{p}(x) = 0.01 + \exp(m(x)), \quad m \sim \mathcal{N}(0, (\delta I - \gamma \Delta)^{-\alpha})$$

(Darcy flow)

Configurations (Darcy flow)

- Domain: $\Omega = [0, 1] \times [0, 1]$.
- Boundary data:

$$\begin{aligned}u_0(x) &= 1 - x_1, \quad x \in \Gamma_{\text{left}} \cup \Gamma_{\text{right}} =: \Gamma_D, \\g(x) &= 0, \quad x \in \Gamma_{\text{top}} \cup \Gamma_{\text{bottom}} := \Gamma_N.\end{aligned}$$

- Source term:

$$f(x) = \sum_{i=1}^9 100 \exp \left(- \left(\frac{\|x - c_i\|_2}{w} \right)^2 \right), \quad (1)$$

where $c_i = (m/4, n/4)$ for $m, n = 1, 2, 3$ and $w = 1/32$.

- Finite element discretization:

$$\begin{aligned}\text{num_x} \times \text{num_y} &= 128 \times 128, \quad \mathbf{p} \in \text{CG}_1, \quad [\sigma, u] \in \text{RT}_1 \times \text{CG}_2, \\ \Rightarrow \# \text{DoFs}(\mathbf{p}) &= 16,641, \quad \# \text{DoFs}([\sigma, u]) = 230,401.\end{aligned}$$

Problem setup (stationary elasticity equation)

Strong form

$$\begin{cases} -\operatorname{div}(\sigma(u; \mathbf{p})) = f, & \text{in } \Omega, \\ u = u_0, & \text{on } \Gamma_D, \\ \sigma(u; \mathbf{p}) \cdot n = t, & \text{on } \Gamma_N. \end{cases}$$

Here, we use σ to denote the first Piola-Kirchhoff stress tensor (instead of the Cauchy stress tensor for notation consistency), and t is the traction. Let

$$F := I + \nabla u, \quad \varepsilon := \frac{1}{2}(F + F^\top - I_d), \quad J := \det(F).$$

$$\text{Lamé parameters: } \lambda := \frac{\mathbf{p}\nu}{(1+\nu)(1-2\nu)}, \quad \mu := \frac{\mathbf{p}}{2(1+\nu)}.$$

- Linear: $\psi(F) = \mu \varepsilon : \varepsilon + \frac{\lambda}{2}(\operatorname{tr}(\varepsilon))^2$

Then

$$\sigma(u; \mathbf{p}) = \sigma(F(u)) := \frac{\partial \psi}{\partial F}(F).$$

Configurations (linear elasticity)

- Domain: $\Omega = [0, 2] \times [0, 1]$.
- Boundary data:

$$u_0(x) = (0, 0)^\top, \quad x \in \Gamma_{\text{left}},$$

$$t(x) = \begin{cases} (0, 0)^\top, & x \in \Gamma_{\text{top}} \cup \Gamma_{\text{bottom}}, \\ (0.06 \exp(-(x_2 - 0.5)^2/4), 0.03(1 + x_2/10))^\top, & x \in \Gamma_{\text{right}}, \end{cases}$$

- Source term $f(x) = (0, 0)^\top$.
- Finite element discretization:

$$\begin{aligned} \text{num_x} \times \text{num_y} &= 128 \times 64, \quad \mathbf{p} \in \text{CG}_1, \quad [\sigma, u] \in \text{RT}_1^2 \times \text{CG}_2^2, \\ \Rightarrow \# \text{DoFs}(\mathbf{p}) &= 8,385, \quad \# \text{DoFs}([\sigma, u]) = 230,914. \end{aligned}$$

Configurations (miscellaneous)

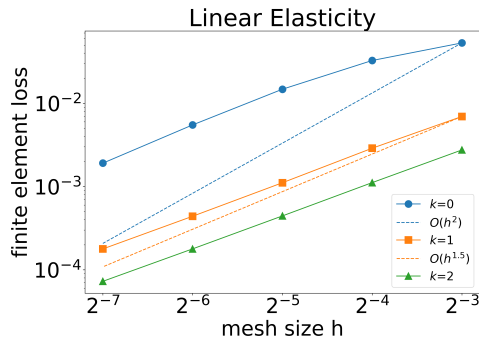
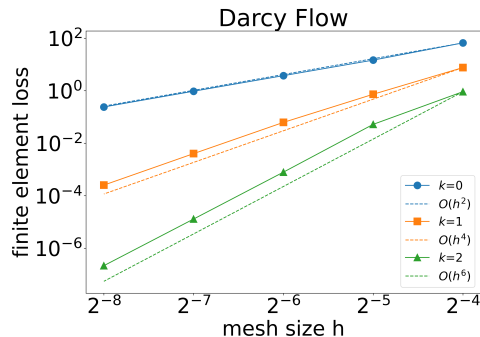
Reduced basis neural operator

- Use $N_{\text{POD}} = 1000$ solution snapshots to compute 512 POD modes
- Precompute reduced weight to speed up training for linear case
- LeakyReLU activation function: $\sigma(x) = \max(0, x) + 0.01 \min(0, x)$
- Initialization method: xavier uniform
- # trainable parameters 15.1 M (Darcy flow) and 6.1 M (elasticity/hyperelasticity)
- Optimizer: SOAP [Vyas et al., 2024]
- Learning rate scheduler: StepLR
- Numerics precision: float32 for CNN, float64 for linear reconstruction

Hardwares and softwares

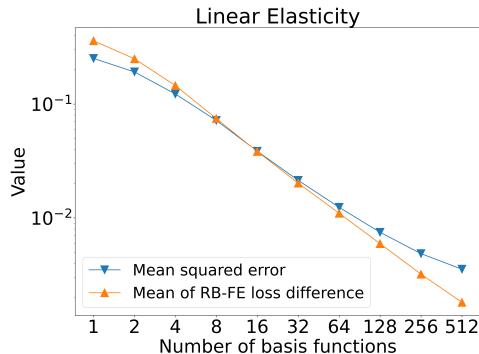
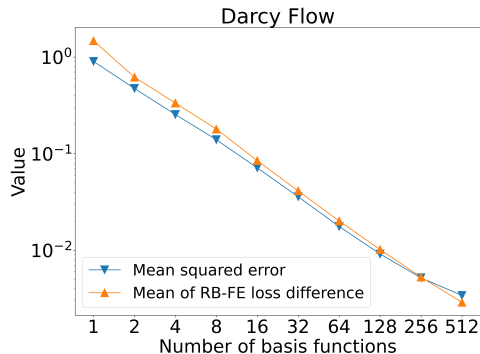
- Device: AMD EPYC 9334 CPUs and one NVIDIA L40S GPU
- Programming language: Python 3.13
- Core libraries: DOLFINx 0.9.0, PyTorch 2.6.0

FE loss vs. mesh size & FE order (with $m = k + 1$ for $\text{RT}_k \times \text{CG}_m$)



$$\begin{aligned} \mathcal{L}([\sigma_h^\circ, u_h^\circ]; \mathbf{p}) &\lesssim h^{2\min(k+1, s_\sigma)} \|\sigma^\circ\|_{H^{s_\sigma}}^2 + h^{2\min(k+1, s_{\text{div}})} \|\text{div } \sigma^\circ\|_{H^{s_{\text{div}}}}^2 + h^{2\min(m, s_u-1)} \|u^\circ\|_{H^{s_u}}^2 \\ &\lesssim h^{2(k+1)}, \quad \text{when } \sigma^\circ \in H^{k+1}, \text{div } \sigma^\circ \in H^{k+1}, u^\circ \in H^{m+1}. \end{aligned}$$

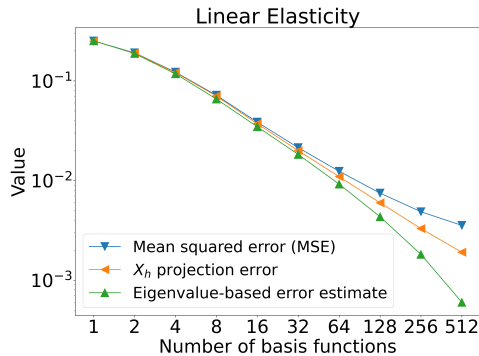
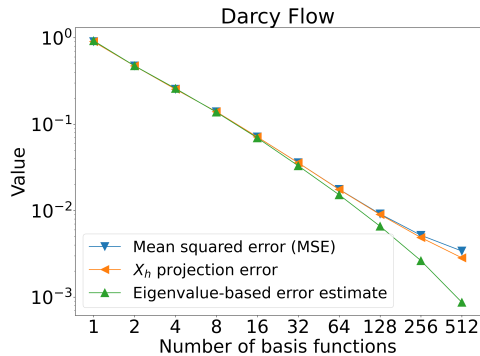
RB error vs. (RB loss - FE loss)



Comparison between the empirical mean squared error $\mathbb{E}_{\mathbf{p} \sim \mu} [\|s_r(\mathbf{p}) - s_h(\mathbf{p})\|_{\mathbb{H}}^2]$ and the empirical mean loss difference $\mathbb{E}_{\mathbf{p} \sim \mu} [\mathcal{L}(s_r(\mathbf{p}); \mathbf{p}) - \mathcal{L}(s_h(\mathbf{p}); \mathbf{p})]$. Both quantities are estimated over 500 random samples (using $\text{RT}_1 \times \text{CG}_2$ elements for s_h).

$$\mathcal{L}(s_r; \mathbf{p}) \approx \|s_h(\mathbf{p}) - s_r\|_{\mathbb{H}}^2 + \mathcal{L}(s_h(\mathbf{p}); \mathbf{p}).$$

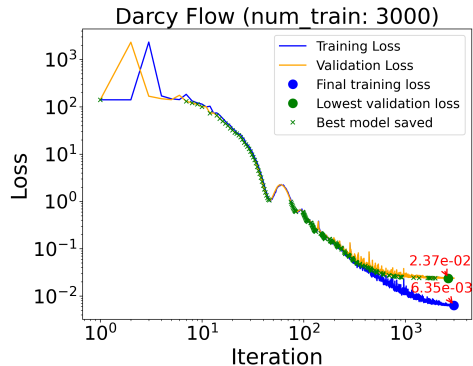
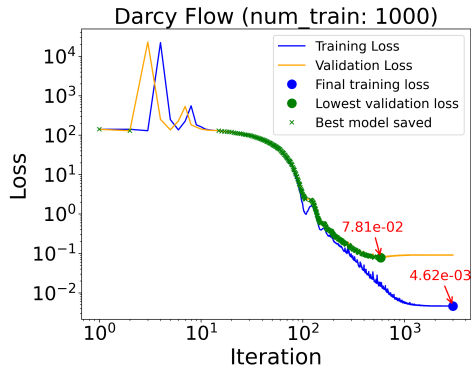
RB FOSLS minimizers' quasi-optimality



Comparison between the empirical mean squared error $\mathbb{E}_{\mathbf{p} \sim \mu} [\|s_r(\mathbf{p}) - s_h(\mathbf{p})\|_{\mathbb{H}}^2]$ of the RB solution s_r (approximated over 500 samples with respect to the FE solution s_h using $\text{RT}_1 \times \text{CG}_2$ elements), the square of the X_h -projection error of s_h onto the RB space $\mathbb{H}_r$, and the eigenvalue-based error estimate.

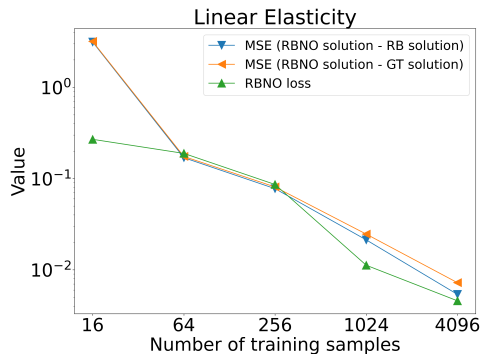
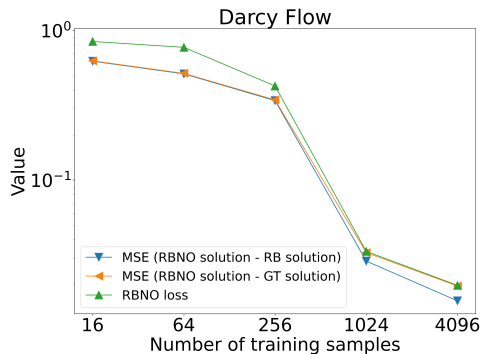
$$\|s_r(\mathbf{p}) - s_h(\mathbf{p})\|_{\mathbb{H}}^2 \approx \min_{t_r \in \mathbb{H}_r} \|t_r - s_h(\mathbf{p})\|_{\mathbb{H}}^2.$$

When to stop training?



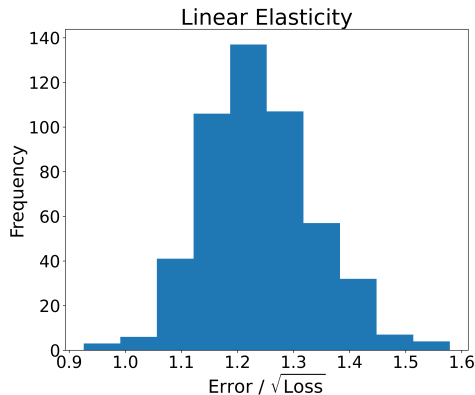
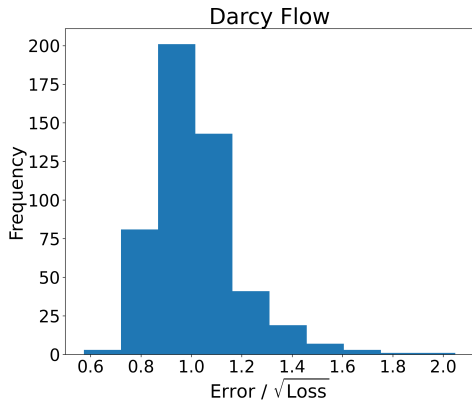
Empirical loss decay over optimization iterations on the training set with 1000 samples (left) and 3000 samples (right), and on the validation set with 500 samples. During training, model checkpoints are updated whenever the validation loss decreases, and the final model is selected as the checkpoint with the lowest validation loss.

MSE & RBNO loss vs. number of training samples



Decay of the empirical RBNO loss $\mathbb{E}_{\mathbf{p} \sim \mu} [\mathcal{L}(s_r(\mathbf{p}; \hat{\theta}); \mathbf{p})]$ at the RBNO solution $s_r(\mathbf{p}; \hat{\theta})$, the empirical mean square error between the RBNO solution and the RB solution $\mathbb{E}_{\mathbf{p} \sim \mu} [\|s_r(\mathbf{p}) - s_r(\mathbf{p}; \hat{\theta})\|_{\mathbb{H}}^2]$, and the empirical mean square error between the RBNO solution and the “ground truth” FE solution (with $\text{RT}_3 \times \text{CG}_4$ elements) $\mathbb{E}_{\mathbf{p} \sim \mu} [\|\bar{s}_h(\mathbf{p}) - s_r(\mathbf{p}; \hat{\theta})\|_{\mathbb{H}}^2]$ with increasing number of training samples from 16 to 64, 256, 1024, and 4096, where the expectation is evaluated with 500 test samples.

Histograms of the ratio $\text{Error}/\sqrt{\text{Loss}}$



Histograms of the ratio $\text{Error}/\sqrt{\text{Loss}}$, between the RBNO error $\|\bar{s}_h(\mathbf{p}) - s_r(\mathbf{p}; \hat{\theta})\|_{\mathbb{H}}$ (with “ground truth” FE solution $\bar{s}_h(\mathbf{p})$ using $\text{RT}_3 \times \text{CG}_4$ elements) and the square root of the RBNO residual loss $\mathcal{L}(s_r(\mathbf{p}; \hat{\theta}); \mathbf{p})$, over 500 test samples for the three problems.

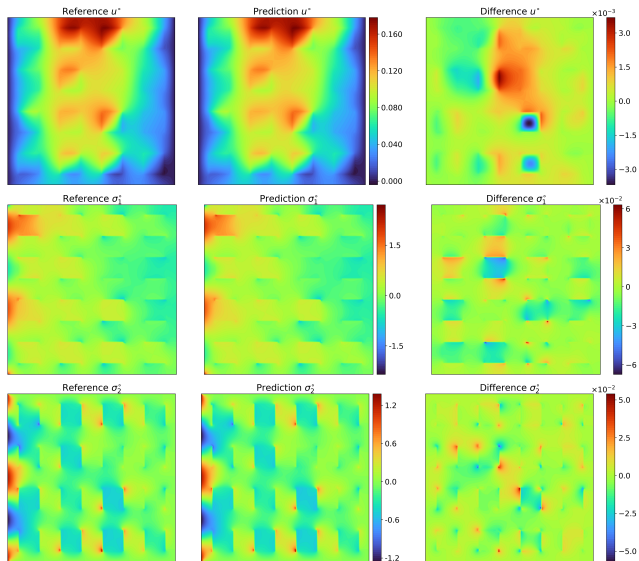
Test relative error and residual loss (PCA-Net, FNO, **RBNO**)

Method	Relative error in $\mathbb{L}$ -norm	Relative error in $\mathbb{H}$ -norm	Residual loss
Heat Conduction			
PCA-Net	1.27×10^{-1} (2.52×10^{-2})	3.32×10^{-1} (5.75×10^{-2})	1.84×10^1 (3.01)
FNO	1.82×10^{-2} (1.66×10^{-3})	1.46×10^{-1} (1.53×10^{-2})	1.90×10^1 (4.12)
RBNO	2.43×10^{-2} (9.35×10^{-3})	1.86×10^{-2} (6.36×10^{-3})	5.33×10^{-4} (1.44×10^{-3})
Darcy Flow			
PCA-Net	1.79×10^{-1} (7.53×10^{-2})	1.13×10^{-1} (2.35×10^{-2})	2.59 (2.03)
FNO	3.30×10^{-2} (1.18×10^{-2})	1.60×10^{-1} (3.81×10^{-2})	5.10 (2.34)
RBNO	6.39×10^{-2} (2.26×10^{-2})	1.23×10^{-2} (3.64×10^{-3})	2.72×10^{-2} (2.61×10^{-2})
Linear Elasticity			
PCA-Net	1.34×10^{-1} (1.03×10^{-1})	3.27×10^{-1} (1.14×10^{-1})	2.46×10^1 (6.11)
FNO	1.17×10^{-2} (3.34×10^{-3})	4.30×10^{-1} (6.11×10^{-2})	2.82×10^1 (6.45)
RBNO	2.49×10^{-2} (5.89×10^{-3})	3.91×10^{-2} (8.68×10^{-3})	4.87×10^{-3} (3.56×10^{-3})

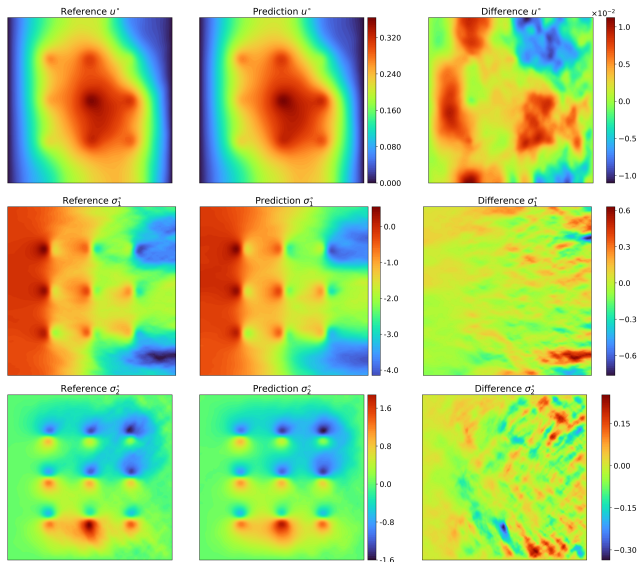
Figure 1: Comparison on the mean (and standard deviation, estimated over 500 test samples) of the relative errors in $\mathbb{L}^2 = L^2 \times L^2$ -norm and $\mathbb{H} = H(\text{div}) \times H^1$ -norm, and residual loss for three neural networks on three problems.

- PCA-Net [Bhattacharya et al., 2021]
- FNO [Li et al., 2020]
- RBNO has similar architecture of POD-DeepONet [Lu et al., 2022]

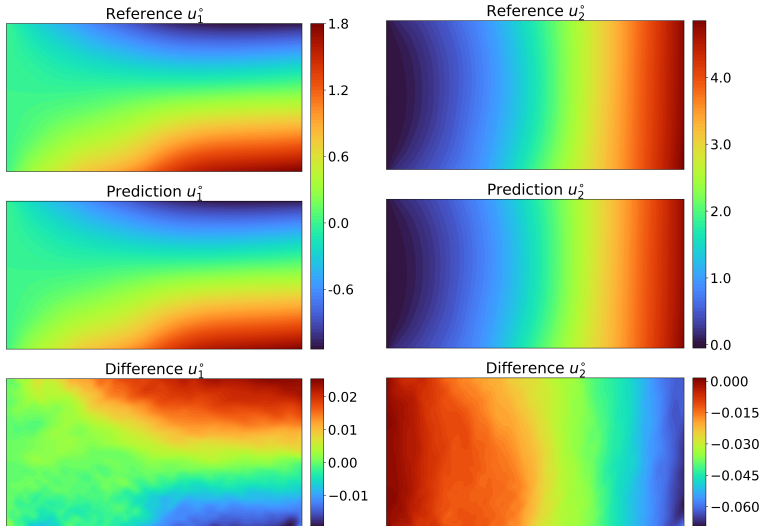
Visualization of ref-pred-diff of u° and σ° (heat conduction)



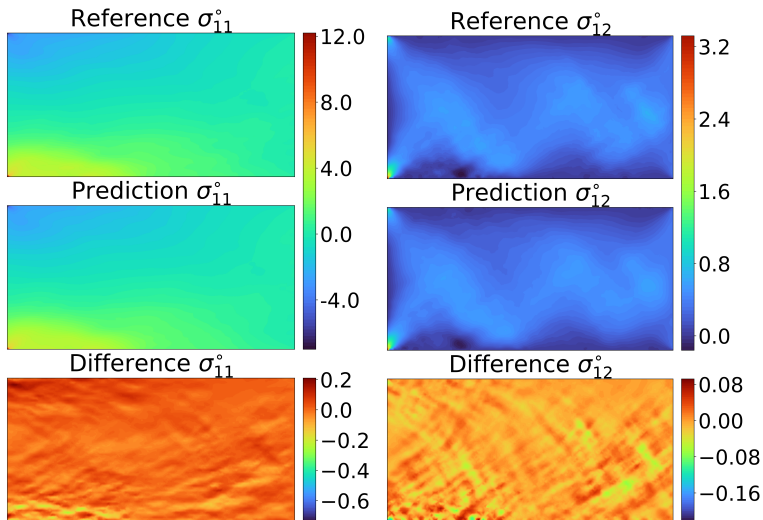
Visualization of ref-pred-diff of u° and σ° (Darcy flow)



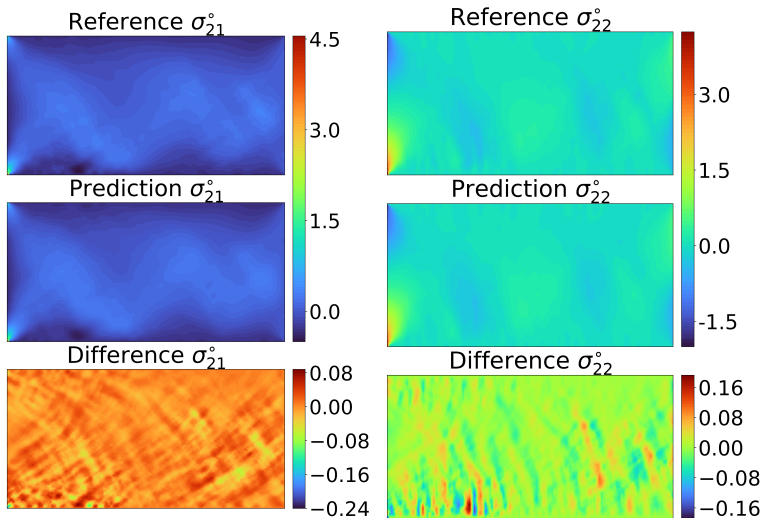
Visualization of ref-pred-diff of u° (linear elasticity)



Visualization of ref-pred-diff of σ_1° (linear elasticity)



Visualization of ref-pred-diff of σ_2° (linear elasticity)



Summary & limitations

Summary

- Develop FOSLS loss in some PDEs with mixed Dirichlet-Neumann BCs
- Loss tracks solution posteriori error
- RBNO outperforms PCA-Net/FNO in $H(\text{div}) \times H^1$ norm

Limitations

- Less favorable when the Kolmogorov n -widths decay slowly
- Assumes boundary conditions do not involve parameters
- Only consider linear PDEs (we are working on nonlinear cases)

Interested? Please see our paper “Variationally correct operator learning: Reduced basis neural operator with a posteriori error estimation” on arXiv.



Paper



Code

Acknowledgement: NSF #2245111, Variationally Stable Neural Networks for Simulation, Learning, and Experimental Design of Complex Physical Systems

Thank you for your attention!

References I



Bachmayr, M., Dahmen, W., and Oster, M. (2025).
Variationally correct neural residual regression for parametric pdes: On the viability of controlled accuracy.
IMA Journal of Numerical Analysis, page draf073.



Bao, G., Ye, X., Zang, Y., and Zhou, H. (2020).
Numerical solution of inverse problems by weak adversarial networks.
Inverse Problems, 36(11):115003.



Bhattacharya, K., Hosseini, B., Kovachki, N. B., and Stuart, A. M. (2021).
Model reduction and neural networks for parametric pdes.
The SMAI journal of computational mathematics, 7:121–157.



Castillo, P. C., Dahmen, W., and Gopalakrishnan, J. (2025).
DPG loss functions for learning parameter-to-solution maps by neural networks.
arXiv preprint arXiv:2506.18773.



Kharazmi, E., Zhang, Z., and Karniadakis, G. E. (2019).
Variational physics-informed neural networks for solving partial differential equations.
arXiv preprint arXiv:1912.00873.



Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. (2020).
Fourier neural operator for parametric partial differential equations.
arXiv preprint arXiv:2010.08895.



Li, Z., Zheng, H., Kovachki, N., Jin, D., Chen, H., Liu, B., Azizzadenesheli, K., and Anandkumar, A. (2024).
Physics-informed neural operator for learning partial differential equations.
ACM/IMS Journal of Data Science, 1(3):1–27.

References II



Lu, L., Meng, X., Cai, S., Mao, Z., Goswami, S., Zhang, Z., and Karniadakis, G. E. (2022).
A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data.
Computer Methods in Applied Mechanics and Engineering, 393:114778.



Raissi, M., Perdikaris, P., and Karniadakis, G. E. (2019).
Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations.
Journal of Computational physics, 378:686–707.



Vyas, N., Morwani, D., Zhao, R., Kwun, M., Shapira, I., Brandfonbrener, D., Janson, L., and Kakade, S. (2024).
Soap: Improving and stabilizing shampoo using adam.
arXiv preprint arXiv:2409.11321.



Yu, B. et al. (2018).
The deep ritz method: a deep learning-based numerical algorithm for solving variational problems.
Communications in Mathematics and Statistics, 6(1):1–12.



Zang, Y., Bao, G., Ye, X., and Zhou, H. (2020).
Weak adversarial networks for high-dimensional partial differential equations.
Journal of Computational Physics, 411:109409.